Faculty of Sciences Department of Computer Sciences

Master thesis

Procedural Generation of Solvable Cooperative Levels for the Laser Learning Environment

Author:
Charels Hugo
hugo.charels@ulb.be

Supervisors:
Lenaerts Tom
Molinghen Yannick

Academic year 2025-2026

Contents

Chapter 1

Introduction

1.1 Context

Cooperative Multi-Agent Reinforcement Learning (MARL) studies settings in which several agents must learn behaviours whose value appears only at the team level. In such settings, the environment is not a neutral container: it determines which coordination patterns are possible, which ones are necessary, and how difficult they are to discover.

This dependence on environment structure is especially strong in sparse-reward cooperative tasks. When reward is issued only after the team objective has been achieved, a training level is useful only if it exposes a meaningful coordination challenge while remaining actually solvable. Unsolvable levels provide no valid signal, and levels that admit independent solutions fail to test the cooperative mechanism they are meant to study.

1.2 Motivation

In cooperative environments whose mechanics create explicit inter-agent dependencies, level design is particularly challenging. A useful training level should not merely appear cooperative — it should provably contain the intended coordination structure.

Procedural Content Generation (PCG) offers a way to scale level creation beyond what manual design allows, but only if the generated instances satisfy the properties that matter for training. Two properties are central. First, a level must be **solvable**. Second, success should **require cooperation** in the specific sense induced by the environment's mechanics. Without those guarantees, generation risks producing levels that are invalid, trivial, or misaligned with the training objective.

1.3 Research Questions and Scope

This thesis addresses the following question: how can we automatically generate levels for a cooperative MARL environment that are provably solvable and provably require the target cooperative interaction?

More precisely, the work is organised around five research questions:

- **RQ1:** How can we formally verify that a level is solvable by the agents?
- **RQ2:** How can we formally verify that solving a level genuinely requires cooperation between agents, rather than allowing independent solutions?
- **RQ3:** How can formal verification be embedded inside a procedural generator so that every accepted level comes with certified properties?
- **RQ4:** Can agents trained exclusively on procedurally generated levels transfer their behaviour to human-designed levels?
- **RQ5:** Can we control the cooperation structure of generated levels by targeting specific profiles such as asymmetric, mutual, or chain dependencies?

The thesis instantiates this framework on one concrete cooperative environment, focusing on a restricted but explicit subset of its mechanics. The formal model covers agent movement, inter-agent blocking interactions, and a bounded-horizon solvability criterion. Additional environment mechanics are outside the scope of the formal guarantees developed here. The broader methodology — coupling procedural generation with a formal verification oracle — is environment-agnostic: it applies to any setting where the target properties are expressible as decision problems.

1.4 Contributions

This thesis makes the following contributions:

- **A decision procedure for bounded-horizon solvability.** We provide a propositional encoding of the solvability decision problem over a bounded time horizon. The procedure either returns a certificate — a valid joint trajectory — or proves that no such trajectory exists within the chosen horizon (Section 4.1.1).
- **A formal cooperation detector.** We define a strict variant of the environment semantics in which agents cannot exploit same-agent blocking to bypass constraints imposed by others. A level requires cooperative interaction if and only if it is solvable under standard semantics and unsolvable under the strict variant (Section 4.1.3).
- **A solver-in-the-loop generation framework.** Building on the solver and cooperation detector, we implement six generators across two property families — solvable and cooperative. Each accepted level is certified by the solver to satisfy the advertised property (Section 4.1.4).
- **A cooperation profile taxonomy and profile-targeted generation.** We define a set of cooperation profiles — asymmetric, mutual, chain, distributed, fully coupled — that characterise the dependency structure of cooperative levels. Generators can target a specific profile, producing levels whose cooperation type is controlled, not merely certified (Section 4.1.3, Section 4.1.4).
- **An empirical evaluation.** We compare two alternative propositional encodings of the agent-uniqueness constraint, measuring their effect on formula size and solver runtime. We also measure generator acceptance rates and cooperation profile distributions across grid sizes (Section 4.1.2, Section 4.2).

1.5 Thesis Structure

The remainder of this thesis is organised as follows.

Chapter 2 positions the work relative to cooperative MARL benchmarks, procedural content generation, and compilation-based planning literature. Chapter 3 introduces the environment model and formalises bounded-horizon solvability and the cooperation requirement. Chapter 4 presents the experimental work in two parts: Part 1 develops the original contribution — the SAT encoding, the cooperation detector, the cooperation profile taxonomy, and the generator family; Part 2 reports the results of the evaluation experiments. Chapter 5 concludes with a summary and future work.

Chapter 2

State of the Art

2.1 Cooperative MARL and Coordination-Critical Benchmarks

This thesis is motivated by a benchmark-design question within cooperative Multi-Agent Reinforcement Learning (MARL): how can one generate training instances whose coordination structure is both non-trivial and formally controlled?

In the fully cooperative setting, all agents optimise a shared return, so the quality of the environment matters as much as the quality of the learning algorithm. If the environment admits only trivial solutions, it does not meaningfully test coordination. If it contains unsolvable instances, it provides no useful training signal at all. For the present thesis, the central issue is therefore not MARL in general, but the design of instances in which inter-agent dependence is both structurally present and formally decidable.

2.2 The Laser Learning Environment

The Laser Learning Environment (LLE) was introduced precisely to study coordination-critical multi-agent tasks [1]. The paper identifies three properties that make the benchmark difficult for value-based MARL methods: **perfect coordination**, **interdependence**, and **zero-incentive dynamics**. Together, these properties create bottlenecks in which one agent must perform a locally unrewarded action that enables another agent to progress.

This benchmark framing is directly relevant to the present work. The thesis does not attempt to improve MARL training algorithms on LLE. Instead, it addresses an upstream question left open by the benchmark paper: how can we generate LLE levels that are guaranteed to be solvable and that contain the beam-blocking dependency on which the benchmark relies?

The LLE paper therefore plays two roles in this thesis. First, it justifies why LLE is an interesting target domain. Second, it provides the conceptual vocabulary used here to discuss cooperation: the key object is not generic teamwork, but a concrete interdependence mechanism created by coloured lasers and same-colour blocking.

2.3 Procedural Generation Under Structural Constraints

Procedural Content Generation (PCG) is useful in reinforcement-learning settings because it can replace a small fixed benchmark set with a larger and more diverse stream of instances [2]. However, generic PCG is not sufficient for the present problem. The difficulty is not merely to produce varied levels, but to produce levels that satisfy logically defined properties.

This distinction matters. A constructive or search-based generator may bias generation toward interesting layouts, but without a verifier it cannot certify that a sampled level is solvable or that success genuinely depends on cooperation. For that reason, the present thesis adopts a constraint-aware view of PCG: generation is coupled to a formal decision procedure, and the solver acts as an acceptance oracle rather than as a post-hoc descriptive tool.

2.4 Compilation-Based Multi-Agent Path Finding

The closest methodological precedent is not PCG for MARL, but compilation-based Multi-Agent Path Finding (MAPF). In standard MAPF, agents move on a discrete graph from start vertices to goal vertices while avoiding collisions. The computational difficulty comes from the interaction between multiple agents and the optimality criterion imposed on the solution.

Surynek's survey [3] shows that MAPF has become a major testbed for compilation-based solving. Instead of searching directly in the original state space, one reduces a MAPF instance to a target formalism such as CSP, SAT, or MILP, then relies on the target solver to handle the combinatorial burden. The survey is especially relevant here for two reasons.

First, it demonstrates that SAT-based reductions are a mature and credible way to solve structured multi-agent planning problems. Second, it makes clear that compilation is not a black-box slogan: modeling choices, encoding size, and the interaction between the source problem and the target solver all matter materially for performance.

The present thesis inherits this compilation perspective. Bounded-horizon LLE solvability is treated as a decision problem and is reduced to SAT. The difference is that LLE is not standard MAPF: the environment contains colour-dependent laser semantics and the property of interest is not path optimality, but solvability and cooperation under the benchmark mechanics.

2.5 SAT-Based MAPF Encoding Design

Beyond the general survey, the MAPF literature also provides concrete lessons about SAT encoding design. The paper by Frommknecht and Surynek [4] studies SAT-based MAPF solving under the makespan objective using an MDD-SAT formulation and compares different solver-facing choices, including eager versus lazy encodings and the use of informative initial assignments.

That paper is relevant to the present thesis not because it solves the same problem, but because it shows that SAT-based multi-agent solving is sensitive to representation details. Performance is not determined only by the underlying decision problem; it also depends on how the problem is encoded and on how the resulting CNF interacts with the chosen SAT solver. This is directly aligned with the experimental part of the current thesis, where two alternative uniqueness encodings are compared empirically.

At the same time, the distance between the two settings should be stated explicitly. Standard MAPF encodings reason about graph motion and collisions. The current thesis must additionally encode time-dependent laser propagation, same-colour immunity, and a strict counterfactual semantics used to define cooperation. The MAPF literature therefore supplies a methodological template, not a drop-in solution.

2.6 Positioning of the Thesis

The literature leaves a clear opening for the present work.

- The LLE paper [1] establishes the benchmark and explains why its coordination bottlenecks are difficult for MARL algorithms, but it does not provide a formal generator for certified cooperative levels.
- The MAPF compilation survey [3] shows that SAT is an effective backend for multi-agent planning problems, but it addresses standard MAPF rather than LLE-specific laser dynamics.
- The MAPF SAT-engineering paper [4] shows that encoding design affects solver performance in practice, but it remains within the standard MAPF framework and does not address cooperation as a semantic property of the instance.

This thesis sits at the intersection of those lines of work. It transfers the compilation-based SAT mindset from MAPF into the LLE setting, formalises bounded-horizon solvability for an LLE-specific model, and introduces a strict-semantics counterfactual that turns a benchmark-level intuition about cooperation into a decidable property inside procedural generation.

Chapter 3

Method

3.1 The Laser Learning Environment

The Laser Learning Environment (LLE) [1] is a grid-based cooperative multi-agent benchmark designed to study coordination-critical tasks. A level consists of a rectangular grid containing walls, coloured laser sources, agent start positions, and exit tiles. Each agent is assigned a colour and must reach its designated exit tile.

The central mechanic is the laser beam. Each source emits a directional beam that propagates cell by cell until it reaches a wall or the grid boundary. An agent whose colour matches the beam is *immune* to it: it may occupy a cell the beam traverses without being harmed, but its physical presence truncates the beam at that cell. Agents of other colours cannot occupy a cell crossed by an active beam.

This truncation is the source of the cooperative structure studied in this thesis. By occupying a position along its own beam, an agent shortens the beam and opens cells beyond it for teammates who are not immune to that colour. Crucially, this action is locally unrewarded: the helper agent receives no direct benefit from blocking its own beam. The reward is issued only when all agents simultaneously occupy their exits, which requires the full team to coordinate successfully.

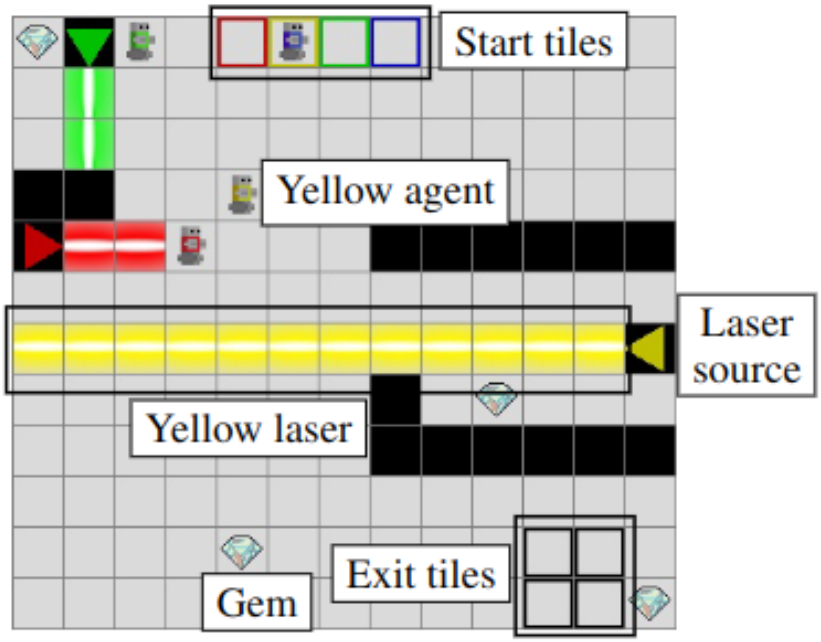

Figure 1: An annotated LLE level. Coloured agents must reach their matching exit tiles. Laser beams block movement for non-immune agents; an immune agent can truncate its own beam by occupying a cell along its path, opening a route for a teammate.

A complete description of the benchmark, its difficulty properties, and its relation to existing MARL methods was given in Chapter 2. The present chapter focuses on the formal model used to reason about solvability and cooperation within a bounded time horizon.

3.2 Boolean Satisfiability

Boolean Satisfiability (SAT) is the problem of deciding whether a propositional formula has a satisfying assignment — a mapping from Boolean variables to true or false that makes the formula evaluate to true. It is the canonical NP-complete decision problem [5].

In practice, SAT solvers operate on formulas in *Conjunctive Normal Form* (CNF). A CNF formula is a conjunction of *clauses*, where each clause is a disjunction of *literals* and a literal is either a variable x or its negation $\neg x$. For example:

$$(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (x_2 \vee x_3)$$

is a CNF formula over three variables. It is satisfied by setting $x_1 = \text{true}$, $x_2 = \text{false}$, $x_3 = \text{true}$, among other assignments.

Modern SAT solvers based on the DPLL procedure and Conflict-Driven Clause Learning (CDCL) can handle industrial instances with millions of variables and clauses [6]. Given a CNF formula, a solver either:

- returns a *satisfying assignment* that witnesses the formula is satisfiable (SAT), or
- certifies that *no* satisfying assignment exists (UNSAT).

The UNSAT certificate is as informative as the SAT witness: it proves the non-existence of a solution. This property is central to the cooperation detection mechanism developed in Section 4.1.3, where UNSAT under a modified encoding is used to certify that no solution exists without the cooperative interaction.

In this thesis, propositional variables are introduced to represent agent positions, laser beam states, and laser activity at each time step. The constraints of the bounded-horizon solvability problem are then encoded as CNF clauses, and a SAT solver is used as a decision oracle. The solver used throughout is Minisat22, accessed through the PySAT interface [7].

3.3 Problem Formalization

We now define the semantic objects and decision problems used throughout the thesis. Two properties are central. **Solvability** asks whether there exists a valid joint trajectory that brings all agents to their exits within a bounded horizon. **Cooperation requirement** asks whether every such trajectory must rely on the same-colour beam-truncation mechanism. Both are formally decidable, and both are certified by the solver for every accepted level.

This section operates at the level of the game semantics; the SAT encoding of these objects follows in Section 4.1.1.

The LLE environment supports between 1 and 4 agents. Accordingly, throughout this thesis we assume that $1 \leq n_a \leq 4$.

Definition 3.1 (LLE Level)

An LLE level is a tuple $L = (H, W, C, s, \mathcal{W}, \mathcal{S}, \mathcal{E})$ where:

- $H, W \in \mathbb{N}^+$ are the height and width of the grid.
- $P = \{(x, y) \mid 0 \leq x < W, 0 \leq y < H\}$ is the set of all grid positions.
- $C = \{0, 1, ..., n_a - 1\}$ is the set of agent colours, with $1 \leq n_a \leq 4$.
- $s : C \to P$ assigns an initial position to each agent, and s is injective.
- $D = \{N, S, E, W\}$ is the set of laser directions.
- $\mathcal{W} \subseteq P$ is the set of wall positions.
- $\mathcal{S} \subseteq C \times D \times P$ is the set of laser sources. A source $(c, d, p) \in \mathcal{S}$ emits a laser of colour c in direction d from position p.
- $\mathcal{E} \subseteq P$ is the set of exit positions, with $|\mathcal{E}| = |C|$.
- In the instances studied in this thesis, each colour appears in at most one laser source.
- The sets $s(C)$, $\mathcal{W}$, $\mathcal{E}$, and

$$P_{\text{src}} = \{p \in P \mid \exists c \in C, d \in D : (c, d, p) \in \mathcal{S}\}$$

  are pairwise disjoint.

We also write

$$C_{\text{src}} = \{c \in C \mid \exists d \in D, p \in P : (c, d, p) \in \mathcal{S}\}$$

for the set of colours that actually have a source.

Definition 3.2 (Active Beams)

Fix a time step t and a joint position map $p_t : C \to P$.

For a source $(c, d, p_s) \in \mathcal{S}$, consider the grid ray starting at p_s and extending in direction d.

- Under the **standard beam semantics**, the beam of source (c, d, p_s) is active on every cell of that ray up to, but not including, the first wall or the first cell occupied by agent c.
- Under the **strict beam semantics**, the beam of source (c, d, p_s) is active on every cell of that ray up to, but not including, the first wall.

A laser of colour c is active at a position when the beam of the unique source of colour c is active there.

Definition 3.3 (Valid Joint Trajectory)

A **horizon** $T_{\max} \in \mathbb{N}$ is the maximum number of joint moves allowed in a candidate solution.

A **joint trajectory** of length $T_{\max}$ is a sequence of joint positions $\sigma = \left(p_0, p_1, ..., p_{T_{\max}}\right)$ where each $p_t : C \to P$ assigns a position to every agent at time step t.

A joint trajectory is **valid** if it satisfies the following conditions:

- **Initialization:** $p_0(c) = s(c)$ for all $c \in C$.
- **Movement:** For all $t < T_{\max}$ and all $c \in C$, $p_{t+1}(c)$ is reachable from $p_{t(c)}$ in one step: the agent may stay in place or move to a 4-neighbouring cell, but it may not move into a wall or a laser-source cell.
- **No collision:** $p_{t(c_1)} \neq p_{t(c_2)}$ for all t and all distinct $c_1, c_2 \in C$.
- **Conservative hand-off rule:** For all $t < T_{\max}$ and all distinct $c_1, c_2 \in C$, agents may not enter a cell occupied by another agent at the previous time step; that is, $p_{t+1}(c_1) \neq p_{t(c_2)}$ and $p_{t(c_1)} \neq p_{t+1}(c_2)$.
- **Laser safety:** No agent c_1 occupies a cell at time t where a laser of colour $c_2 \in C_{\text{src}}$, with $c_2 \neq c_1$, is active under the standard beam semantics of Definition 3.2.
- **Stay on exit:** If $p_{t(c)} \in \mathcal{E}$ for some $t < T_{\max}$, then $p_{t+1}(c) = p_{t(c)}$.

Definition 3.4 (Solvability)

A level L is **solvable** with horizon $T_{\max}$ if there exists a valid joint trajectory σ of length $T_{\max}$ such that the set of occupied positions at time $T_{\max}$ is exactly the set of exits:

$$\left\{p_{T_{\max}}(c) \mid c \in C\right\} = \mathcal{E}$$

A level is **solvable** without qualification if it is solvable for some finite horizon.

The restriction to a bounded horizon is natural for the SAT encoding. In the model studied here, beam activity at time t is a deterministic function of the joint position map p_t. Hence the full state is determined by the joint agent positions. If a trajectory repeats the same joint position twice, the intervening segment forms a loop and can be removed without affecting reachability of later states. Therefore, if a level is solvable at all, it is solvable within a finite horizon bounded by the number of collision-free joint configurations.

Scope note. All formal guarantees in this thesis are stated relative to a fixed finite horizon $T_{\max}$. The solver decides whether a level is solvable within that horizon, not whether it is solvable under an unbounded notion of play. In practice, $T_{\max}$ is set to a value known to be sufficient for the instances

of interest; the choice of horizon is a parameter of the generation process, not a formal limitation of the approach.

Definition 3.5 (Bounded-Horizon LLE Solvability Problem)

The **bounded-horizon LLE solvability problem** is the following decision problem.

- **Input:** an LLE level L and a horizon $T_{\max} \in \mathbb{N}$.
- **Question:** does L admit a valid joint trajectory of length $T_{\max}$ whose final occupied positions are exactly the exits?

Definition 3.6 (Strict Beam Semantics)

A **strict trajectory** of length $T_{\max}$ is defined exactly as in Definition 3.3, except that beam activity is computed using the strict beam semantics of Definition 3.2: same-colour occupancy no longer truncates the corresponding beam.

In the model studied here, the relevant cooperative action is precisely this same-colour beam-truncation mechanism: an agent occupies a position that would otherwise allow its own beam to continue, thereby opening a path for another agent.

Definition 3.7 (Cooperation Requirement with Horizon $T_{\max}$)

A level L is said to **require cooperation** with horizon $T_{\max}$ if:

- L is solvable under the standard semantics.
- L admits no strict trajectory of length $T_{\max}$ whose final positions occupy all exit tiles.

When the horizon is clear from context, we simply say that L requires cooperation. This bounded form is the one used throughout the rest of the methods chapter, since both the SAT solver and the cooperation detector operate at a fixed finite horizon.

Chapter 4

Experiments

4.1 Part 1 — Contribution

This part presents the original technical contribution of the thesis. Chapter 3 established the formal problem setting: the environment model, the two decision problems, and the semantic objects needed to state them precisely. Building on that foundation, the following sections introduce the SAT encoding of those decision problems, the cooperation detector based on a strict counterfactual semantics, and the family of procedural generators that use the resulting decision procedures as acceptance oracles.

The logical order is important. The cooperation detector is not an isolated add-on; it depends on the standard solvability encoding and changes only the beam semantics needed to test whether the blocking action is genuinely necessary. The generator family then reuses those two decision procedures to certify the advertised properties of accepted levels.

4.1.1 Solver by Reduction to SAT

4.1.1.1 Definition of Sets

We reuse the level objects introduced in Section 3.3: the grid dimensions H and W, the position set P, the colour set C, the direction set D, the wall set $\mathcal{W}$, the source set $\mathcal{S}$, the exit set $\mathcal{E}$, and the start map s.

The SAT reduction introduces a finite horizon $T_{\max} \in \mathbb{N}$, which is the maximum number of joint moves allowed in the bounded decision problem, together with the discrete time-step set $T = \{0, 1, ..., T_{\max}\}$.

The sets P_{src} and C_{src} are as defined in Section 3.3. We recall that each colour appears in at most one laser source; under this assumption, when a source of colour c exists, its position and direction are uniquely determined by c. We write $s(c)$ for the initial position of agent $c \in C$, as defined in Definition 3.1.

4.1.1.2 Definition of Variables

We introduce three families of propositional variables.

- $a_{c,x,y,t}$: true iff agent $c \in C$ occupies position $(x, y) \in P$ at time step $t \in T$.
- $b_{c,d,x,y,t}$: true iff the beam emitted by the source $(c, d, p_s) \in \mathcal{S}$ is active at position $(x, y) \in P$ at time $t \in T$.
- $l_{c,x,y,t}$: true iff a laser of colour $c \in C_{\text{src}}$ is active at position $(x, y) \in P$ at time $t \in T$.

4.1.1.3 Constraints and Logical Encoding

We now formalise the constraint families used in the reduction. Each group of clauses corresponds to one logical component of the bounded-horizon decision problem.

1. **Initialization**

   - **Agents:**
     Each agent $c \in C$ is placed at its designated starting position $s(c)$ at $t = 0$; all other positions are unoccupied by that agent:

$$\bigwedge_{c \in C} \bigwedge_{(x,y) \in P} \begin{cases} a_{c,x,y,0} & \text{if } (x, y) = s(c) \\ \neg a_{c,x,y,0} & \text{otherwise} \end{cases}$$

   - **Laser sources:**
     For each laser source $(c, d, (x_s, y_s)) \in \mathcal{S}$, the beam is always active at its origin, at every time step:

$$\bigwedge_{(c,d,(x_s,y_s)) \in \mathcal{S}} \bigwedge_{t \in T} b_{c,d,x_s,y_s,t}$$

2. **Agent Movements**

   We define the set of positions reachable from (x, y) in one step as (x, y) itself together with its four grid neighbours, excluding walls and laser-source positions:

$$\begin{aligned} \text{next}(x, y) = \{(x', y') \in \{(x, y), (x, y-1), (x+1, y), (x, y+1), (x-1, y)\} \mid \\ (x', y') \in P, (x', y') \notin \mathcal{W}, (x', y') \notin P_{\text{src}}\} \end{aligned}$$

   - **Forward consistency:**
     If agent c is at position (x, y) at time t, it must be at some position in $\text{next}(x, y)$ at time $t + 1$:

$$\bigwedge_{c \in C} \bigwedge_{t=0}^{T_{\max}-1} \bigwedge_{(x,y) \in P} a_{c,x,y,t} \rightarrow \bigvee_{(x',y') \in \text{ next}(x,y)} a_{c,x',y',t+1}$$

$$\Updownarrow$$

$$\bigwedge_{c \in C} \bigwedge_{t=0}^{T_{\max}-1} \bigwedge_{(x,y) \in P} \neg a_{c,x,y,t} \vee \bigvee_{(x',y') \in \text{ next}(x,y)} a_{c,x',y',t+1}$$

   Two methods are proposed to enforce that each agent occupies at most one position at each time step.

- **Global uniqueness:**
  No two distinct positions can both be occupied by agent c at time t:

$$\bigwedge_{c\in C}\bigwedge_{t\in T}\bigwedge_{(x_1,y_1)\in P}\bigwedge_{\substack{(x_2,y_2)\in P,\\(x_2,y_2)\neq(x_1,y_1)}} \neg a_{c,x_1,y_1,t} \vee \neg a_{c,x_2,y_2,t}$$

- **Local uniqueness:**
  No two distinct positions in $\text{next}(x, y)$ can simultaneously be occupied by agent c at time $t + 1$:

$$\bigwedge_{c\in C}\bigwedge_{t=0}^{T_{\max}-1}\bigwedge_{(x,y)\in P}\bigwedge_{(x',y')\in \text{next}(x,y)}\bigwedge_{\substack{(x'',y'')\in \text{next}(x,y),\\(x'',y'')\neq(x',y')}} \neg a_{c,x',y',t+1} \vee \neg a_{c,x'',y'',t+1}$$

  Backward consistency:
  If agent c is at position (x, y) at time $t + 1$, it must have been at some position in $\text{next}(x, y)$ at time t:

$$\bigwedge_{c\in C}\bigwedge_{t=0}^{T_{\max}-1}\bigwedge_{(x,y)\in P} a_{c,x,y,t+1} \rightarrow \bigvee_{(x',y')\in \text{next}(x,y)} a_{c,x',y',t}$$

$$\Updownarrow$$

$$\bigwedge_{c\in C}\bigwedge_{t=0}^{T_{\max}-1}\bigwedge_{(x,y)\in P} \neg a_{c,x,y,t+1} \vee \bigvee_{(x',y')\in \text{next}(x,y)} a_{c,x',y',t}$$

- **No simultaneous occupation:**
  Two distinct agents cannot share the same position at the same time, nor can they move to a position already occupied by the other agent:

$$\bigwedge_{c_1\in C}\bigwedge_{c_2\in C,c_2\neq c_1}\bigwedge_{(x,y)\in P}\bigwedge_{t\in T} \neg a_{c_1,x,y,t} \vee \neg a_{c_2,x,y,t}$$

$$\bigwedge_{c_1\in C}\bigwedge_{c_2\in C,c_2\neq c_1}\bigwedge_{(x,y)\in P}\bigwedge_{t=0}^{T_{\max}-1} \neg a_{c_1,x,y,t+1} \vee \neg a_{c_2,x,y,t}$$

$$\bigwedge_{c_1\in C}\bigwedge_{c_2\in C,c_2\neq c_1}\bigwedge_{(x,y)\in P}\bigwedge_{t=0}^{T_{\max}-1} \neg a_{c_1,x,y,t} \vee \neg a_{c_2,x,y,t+1}$$

- **Victory condition:**
  Each exit must be occupied by at least one agent at time $T_{\max}$. Since $|\mathcal{E}| = n_a$ and agents cannot share positions, this enforces a bijection between agents and exits:

$$\bigwedge_{(x,y)\in\mathcal{E}}\bigvee_{c\in C} a_{c,x,y,T_{\max}}$$

- **Stay on exit:**
  Once an agent reaches an exit, it remains there for all subsequent time steps:

$$\bigwedge_{c\in C}\bigwedge_{(x,y)\in\mathcal{E}}\bigwedge_{t=0}^{T_{\max}-1} \neg a_{c,x,y,t} \vee a_{c,x,y,t+1}$$

3. **Laser Activity**

We define $\text{next}_{d(x,y)}$ as the position immediately adjacent to (x, y) in direction d:

$$\text{next}_{d(x,y)} = \begin{cases} (x, y-1) \text{ if } d = N \\ (x+1, y) \text{ if } d = E \\ (x, y+1) \text{ if } d = S \\ (x-1, y) \text{ if } d = W \end{cases}$$

- **Walls block beams:**
  A beam cannot be active at a wall position:

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{(x,y)\in\mathcal{W}} \bigwedge_{t\in T} \neg b_{c,d,x,y,t}$$

- **Beam propagation:**
  Beam propagation clauses are instantiated only when the successor cell $(x', y') = \text{next}_{d(x,y)}$ lies inside the grid, is not a wall, and is not itself a source cell. Source cells are handled separately by the initialization clauses above. Under these conditions, the beam is active at (x', y') iff it is active at (x, y) and no agent of colour c occupies (x', y'):

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{t\in T} \bigwedge_{\substack{(x,y)\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\notin P_{\text{src}}}} b_{c,d,x',y',t} \leftrightarrow \left(b_{c,d,x,y,t} \wedge \neg a_{c,x',y',t}\right)$$

$$\Updownarrow$$

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{t\in T} \bigwedge_{\substack{(x,y)\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\notin P_{\text{src}}}} \neg b_{c,d,x,y,t} \vee a_{c,x',y',t} \vee b_{c,d,x',y',t}$$

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{t\in T} \bigwedge_{\substack{(x,y)\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\notin P_{\text{src}}}} b_{c,d,x,y,t} \vee \neg b_{c,d,x',y',t}$$

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{t\in T} \bigwedge_{\substack{(x,y)\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\notin P_{\text{src}}}} \neg a_{c,x',y',t} \vee \neg b_{c,d,x',y',t}$$

- **Link between beam and laser variables:**
  Since each colour has at most one source in the instances considered here, the laser variable $l_{c,x,y,t}$ is true at a position iff the beam of the unique source of colour c is active there:

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{(x,y)\in P} \bigwedge_{t\in T} b_{c,d,x,y,t} \leftrightarrow l_{c,x,y,t}$$

$$\Updownarrow$$

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{(x,y)\in P} \bigwedge_{t\in T} \left(\neg b_{c,d,x,y,t} \vee l_{c,x,y,t}\right) \wedge \left(\neg l_{c,x,y,t} \vee b_{c,d,x,y,t}\right)$$

- **Agents cannot step on active lasers:**
  An agent of colour c_1 cannot occupy a position where an active laser of some source colour $c_2 \in C_{\text{src}}$, with $c_2 \neq c_1$, is present. Agents are immune only to lasers of their own colour:

$$\bigwedge_{c_1 \in C} \bigwedge_{\substack{c_2 \in C_{\text{src}}, \\ c_2 \neq c_1}} \bigwedge_{(x,y) \in P} \bigwedge_{t \in T} l_{c_2,x,y,t} \rightarrow \neg a_{c_1,x,y,t}$$

$$\Updownarrow$$

$$\bigwedge_{c_1 \in C} \bigwedge_{\substack{c_2 \in C_{\text{src}}, \\ c_2 \neq c_1}} \bigwedge_{(x,y) \in P} \bigwedge_{t \in T} \neg l_{c_2,x,y,t} \vee \neg a_{c_1,x,y,t}$$

4.1.1.4 Clause Complexity and Polynomial Size

To show that the reduction has polynomial size, it is enough to bound the number of generated clauses as a function of the input parameters. Let

$$\begin{aligned} n &= |C|, \\ p &= |P| = HW, \\ s &= |\mathcal{S}|, \\ e &= |\mathcal{E}|, \\ \tau &= T_{\max} + 1 \end{aligned}$$

and write

$$V = P \setminus (\mathcal{W} \cup P_{\text{src}})$$

for the walkable cells.

We count clauses rather than literals. The main families admit the following bounds.

- **Initialization.** The agent-initialisation clauses contribute exactly np unit clauses, and the laser-source initialisation contributes exactly $s\tau$ unit clauses.
- **Movement constraints.** Forward consistency contributes at most $n(\tau-1)p$ clauses. The two uniqueness formulations differ here:

  $$\text{global uniqueness} = n(\tau-1)\binom{p}{2}$$

  $$\text{local uniqueness} = n(\tau-1)\sum_{u \in V}\binom{|\text{next}(u)|}{2}$$

  Since $|\text{next}(u)| \leq 5$, the local uniqueness term is bounded by

  $$n(\tau-1)10\ |V| \leq 10n(\tau-1)p$$

  and backward consistency contributes at most $n(\tau-1)p$ further clauses. The collision and conservative hand-off clauses contribute at most

  $$\binom{n}{2}(\tau + 2(\tau-1))p = \binom{n}{2}(3\tau-2)p$$

  The exit condition contributes exactly e clauses, and the stay-on-exit condition contributes exactly $ne(\tau-1)$ clauses.
- **Laser constraints.** The laser-safety clauses contribute at most $ns\tau p$ clauses. Beam propagation contributes at most $3s\tau p$ clauses, because each admissible propagation edge yields either one wall-

blocking clause or three equivalence clauses. The beam/laser linking clauses contribute exactly $2s\tau p$ clauses.

Therefore the total number of clauses is polynomial in n, p, s, and τ. More precisely, with the global formulation the dominant term is

$$O(n\tau p^2 + n^2\tau p + ns\tau p)$$

while with the local formulation it is

$$O(n\tau p + n^2\tau p + ns\tau p)$$

Since each clause is generated by a simple bounded computation inside these loops, the reduction itself is computable in polynomial time as well. This justifies the claim that bounded-horizon LLE solvability is polynomial-time reducible to SAT.

4.1.1.5 Correctness of the Reduction

Proposition 4.8 (Correctness of the SAT Reduction)

Let L be an LLE level and let $T_{\max}$ be a horizon. For either movement formulation described above, the CNF formula $\Phi(L, T_{\max})$ is satisfiable if and only if there exists a valid joint trajectory of length $T_{\max}$ for L.

Proof.

For soundness, assume $\Phi(L, T_{\max})$ is satisfiable. Initialization fixes exactly one start position for each agent at time 0. For the global formulation, forward consistency together with pairwise exclusion ensures by induction on time that each agent occupies exactly one legal position at every later step. For the local formulation, the same conclusion follows from forward consistency, local exclusivity, and backward consistency. We may therefore define a joint trajectory $p_{t(c)}$ by reading off the unique true agent variable for each colour c and time t. The movement clauses enforce legal motion between consecutive steps; the collision clauses enforce both simultaneous separation and the conservative hand-off rule; the laser clauses enforce safety with respect to active beams; and the exit clauses enforce the terminal condition. Hence the extracted trajectory is valid.

For completeness, assume a valid joint trajectory of length $T_{\max}$ is given. Set each $a_{c,x,y,t}$ according to whether agent c occupies (x, y) at time t in the trajectory. Set beam variables $b_{c,d,x,y,t}$ and laser variables $l_{c,x,y,t}$ according to the deterministic beam dynamics induced by the same agent positions. Every clause family is then satisfied by construction: initialization matches the start state, movement and collision clauses match the trajectory semantics, beam propagation follows the induced laser dynamics, and the final positions occupy all exits. Therefore $\Phi(L, T_{\max})$ is satisfiable. □

4.1.1.6 Complexity-Theoretic Consequences

We can now state the consequence for the decision problem introduced in Definition 4.5.

The bounded-horizon LLE solvability problem lies in **NP**. A candidate trajectory can be verified in polynomial time by simulating the joint execution and checking that all agents occupy the exit tiles at the end without violating the movement, collision, and laser constraints defined in Section 3.3.

Combined with the polynomial-time construction above and Proposition 4.8, this shows that bounded-horizon LLE solvability is polynomial-time many-one reducible to SAT:

$$\text{LLE-Solvability} \leq_p \text{SAT}$$

Thus bounded-horizon LLE solvability is **at most as hard as SAT**: any SAT algorithm yields an algorithm for this decision problem with only polynomial overhead from the reduction.

Whether bounded-horizon LLE solvability is also **NP-hard** remains open in the present work. Establishing NP-hardness would require a polynomial-time reduction in the opposite direction, from a known NP-hard problem to LLE solvability. This thesis does not claim such a result.

It is also important to distinguish proved statements from standard complexity-theoretic beliefs. The question whether $\text{P} = \text{NP}$ remains open. Accordingly, statements here about worst-case difficulty should be read only through the formal claims we have established: SAT is NP-complete, bounded-horizon LLE solvability belongs to NP, and the reduction above places bounded-horizon LLE solvability within the polynomial-time many-one image of SAT.

In practice, this positioning explains why a SAT-based approach is attractive: the solver inherits the strong empirical performance of modern CDCL SAT solvers on many structured instances, even though the worst-case complexity remains exponential.

4.1.2 Evaluation Protocol

The first benchmark implemented in this project isolates the SAT solver rather than the generators. Its goal is to compare the two movement formulations used in the encoding: the **local** formulation, which combines neighbourhood-based exclusivity with backward consistency, and the **global** formulation, which enforces uniqueness by pairwise exclusion over the whole grid.

4.1.2.1 Metrics

For each run, the benchmarking code records:

- the total number of clauses in the generated CNF;
- the clause count contributed by each major constraint family;
- the CNF generation time;
- the SAT solving time;
- the total time obtained by summing generation and solving.

The implementation also stores per-constraint method profiles for the movement constraint, which allows us to inspect the part of the final CNF generated inside the movement-constraint module.

4.1.2.2 Solver and Level Sets

All benchmark runs use the same SAT backend as the main solver implementation, namely `Minisat22` through the PySAT interface. The benchmarking script can evaluate the six hand-crafted benchmark levels distributed with the environment, each paired with a horizon known to be sufficient for solvability. It can also benchmark custom levels constructed programmatically.

The experiment reported in Section 4.2 uses four representative levels: three synthetic instances of increasing size and one original benchmark level. This combination exposes both scaling behaviour and the behaviour of the solver on a realistic cooperative puzzle.

4.1.2.3 Protocol

For each level and each movement formulation, the benchmark performs one profiled run to extract the exact clause counts and the full constraint breakdown. It then repeats the same solver invocation for 100 runs, each time on a fresh copy of the world, and reports the mean and standard deviation of generation time and solve time. Using fresh world copies avoids benchmark contamination by mutable environment state.

No timeout or parallel speedup is introduced in this protocol. The measurements should therefore be read as direct comparisons between the two SAT formulations on the same machine, rather than as hardware-independent absolute performance claims.

4.1.2.4 Outputs

The benchmarking pipeline produces a console summary table, a JSON file containing aggregated benchmark statistics, and a set of plots for clause counts, per-constraint clause breakdowns, and timing statistics. The results section (Section 4.2) draws on these outputs to interpret the trade-off between the two movement formulations.

4.1.3 Cooperation Detection

4.1.3.1 Strict SAT Encoding

Recall from Definition 3.6 that strict beam semantics changes only one aspect of the dynamics: same-colour occupancy no longer truncates the corresponding beam. Same-colour immunity is unchanged.

Accordingly, the strict SAT encoding keeps the standard laser-safety clauses and replaces only the same-colour beam-propagation rule. For every source $(c, d, p_s) \in \mathcal{S}$, every admissible propagation edge from (x, y) to (x', y'), and every time step $t \in T$, the strict encoding uses

$$b_{c,d,x',y',t} \leftrightarrow b_{c,d,x,y,t}$$

instead of the standard equivalence

$$b_{c,d,x',y',t} \leftrightarrow \left(b_{c,d,x,y,t} \wedge \neg a_{c,x',y',t}\right).$$

Thus the beam continues through agents of the matching colour instead of stopping at them. We denote the resulting CNF formula by $\Phi_{\text{strict}}(L, T_{\max})$ and the corresponding solver by StrictSolver.

4.1.3.2 Why This Captures Cooperation

Under the LLE mechanics studied here, the cooperative action of interest is for an agent to occupy a cell that would otherwise allow its own beam to continue, thereby making another agent's path safe. Standard solvability allows this beam-truncation mechanism; strict solvability removes it.

Therefore, if a level is solvable under the standard semantics but unsatisfiable under the strict semantics, every successful standard solution must rely on at least one same-colour beam-truncation step.

4.1.3.3 Formal Theorem and Proof

Theorem 4.9 (Cooperation Detection Criterion)

Let L be an LLE level and $T_{\max}$ a time horizon. Then L requires cooperation with horizon $T_{\max}$ if and only if $\Phi(L, T_{\max})$ is satisfiable and $\Phi_{\text{strict}}(L, T_{\max})$ is unsatisfiable.

Proof.

($\rightarrow$) Assume that L requires cooperation with horizon $T_{\max}$. By Definition 3.7, L is solvable under the standard semantics, so $\Phi(L, T_{\max})$ is satisfiable. Suppose for contradiction that $\Phi_{\text{strict}}(L, T_{\max})$ is also satisfiable. Then there exists a strict trajectory whose final positions occupy all exit tiles. Since strict beam semantics differs from the standard one only by removing same-colour beam truncation, such a trajectory is also a valid standard trajectory that succeeds without using that mechanism. This contradicts Definition 3.7. Therefore $\Phi_{\text{strict}}(L, T_{\max})$ is unsatisfiable.

($\leftarrow$) Assume that $\Phi(L, T_{\max})$ is satisfiable and $\Phi_{\text{strict}}(L, T_{\max})$ is unsatisfiable. The first condition implies that L is solvable under the standard semantics. Suppose that some successful standard trajectory used no same-colour beam-truncation step. Then the same joint positions would also satisfy the strict beam semantics, because the only semantic difference between the two models concerns exactly that truncation mechanism. This would yield a satisfying assignment for $\Phi_{\text{strict}}(L, T_{\max})$, contradicting unsatisfiability. Hence every successful standard trajectory must use at least one same-colour beam-truncation step, so L requires cooperation with horizon $T_{\max}$. $\square$

4.1.3.4 Practical Algorithm

The cooperation detector runs two SAT calls on the same level:

1. Run $\text{Solver}(L, T_{\max})$. If the result is UNSAT, the level is unsolvable for that horizon, so it is rejected before cooperation is considered.

2. Run StrictSolver$(L, T_{\max})$. If the result is UNSAT, the level requires cooperation for the same horizon.

Both calls share the same bounded horizon and differ only in the beam-propagation clauses. For benchmark levels, the horizon can be chosen from known solution lengths; for generated levels, it is the user-supplied generation parameter $T_{\max}$.

Scope note. The cooperation notion defined here is intentionally specific: it captures same-colour beam-truncation as the relevant cooperative act. A level that requires two agents to coordinate their movements for unrelated geometric reasons — without any laser blocking being involved — would not be identified as cooperative by this detector. This definition is not claimed to exhaust every possible interpretation of cooperation in multi-agent environments; it is the specific mechanism studied in this benchmark, and the formal guarantee is scoped accordingly.

4.1.4 Level Generators

4.1.4.1 Design Pattern

All generators follow a common architecture built around three principles:

1. **SAT as oracle**: the solver is not post-hoc; it is embedded in the generation loop. A candidate level is accepted only if the solver confirms the desired property (solvability, cooperation).
2. **Separation of concerns**: level construction and property verification are decoupled. Generators build candidate levels using domain-specific heuristics; the solver decides acceptance.
3. **Extensibility**: every generator extends `BaseGenerator` and is registered via the `@register_generator` decorator, making it available to the CLI without modifying core code.

In implementation terms, each generator repeatedly performs the following loop: sample or construct a candidate layout, reject it if it violates generator-specific structural constraints, build an `lle.World`, and finally run the appropriate SAT-based acceptance test. Solvable generators stop after the first candidate certified satisfiable within the target horizon, while cooperative generators add the strict-beam counterfactual test and, optionally, a cooperation-profile filter.

4.1.4.2 Generation Targets

Viewed through the solvability and cooperation definitions of Section 3.3, the generator family targets the three level categories shown in Figure Figure 2. Solvable generators accept levels in categories (b) and (c), cooperative generators accept only levels in category (c), and unsolvable levels in category (a) are always rejected.

(a) Unsolvable
rejected by all generators

(b) Solvable, no cooperation
accepted only by solvable generators

(c) Solvable and cooperative
target of cooperative generators

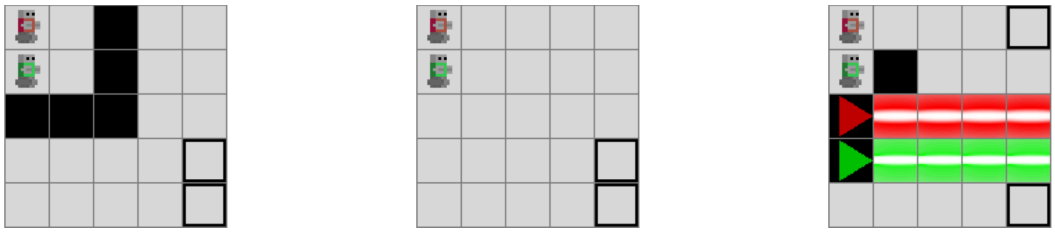

Figure 2: Target level categories for the generator family.

4.1.4.3 Random Solvable Generator

The random solvable generator is the baseline member of the family. It samples pairwise-distinct positions for agent starts, exits, walls, and laser sources uniformly over the grid, assigns a random direction to each source, and submits the resulting world to the solver. A candidate is accepted only if it is satisfiable within the requested horizon $T_{\max}$; when a lower bound $T_{\min}$ is provided, the generator also requires the candidate to be unsatisfiable for $T_{\min} - 1$, thereby selecting levels that fall inside a difficulty window.

This generator is deliberately simple. Its main value is methodological: it gives an unbiased sampling baseline against which more structured generators can be compared. Its main weakness is rejection rate. As the grid grows and the number of interacting entities increases, purely random layouts quickly become dominated by unsolvable or trivial instances.

4.1.4.4 Constrained Random Solvable Generator

A structured variant that biases generation toward solvable configurations before any SAT call is made. Relative to the random solvable generator, it rejects candidates that are already geometrically degenerate, for example when a laser points outside the grid immediately, when a laser would have zero beam length, or when an exit lies on an unavoidable beam segment.

These filters do not themselves prove solvability, but they remove a large class of obviously bad candidates before invoking the solver. The generator therefore remains sound with respect to the formal solvability guarantee, while typically spending less time on layouts that fail for purely local geometric reasons.

4.1.4.5 Random Cooperative Generator

The random cooperative generator extends the random solvable generator with a second SAT test based on the strict semantics of Section 4.1.3. A candidate is accepted only if it is satisfiable under the standard encoding and unsatisfiable under the strict encoding. This guarantees that every accepted level structurally requires the beam-truncation mechanism identified as cooperation in Definition 4.7.

The current implementation augments this binary guarantee with a **cooperation profile analyzer**. The binary detector remains the formal guarantee used throughout the thesis: a level is cooperative if and only if it is satisfiable under the standard semantics and unsatisfiable under the strict semantics. The analyzer adds a second layer whose purpose is to distinguish **which kind** of cooperation the accepted level exhibits.

Given a cooperative level, we first extract a valid joint plan from the standard SAT model. We then run selective counterfactual checks in which one agent at a time loses the same-colour laser interaction used for helping behaviour. If the level becomes unsatisfiable under that selective restriction, the agent is identified as a necessary helper. By combining these counterfactual checks with the helping actions observed in the extracted plan, we build a directed dependency graph between agents.

This dependency graph is used as a generation target. In the present implementation, the generator can recognise and filter levels according to the following profile families:

- **cooperative**: binary cooperation is required, regardless of finer structure;
- **asymmetric**: at least one one-way helping relation is present;
- **mutual**: two agents depend on each other;
- **chain**: dependencies form a directed chain without branching;
- **distributed**: one agent depends on multiple distinct helpers;
- **fully coupled**: all agents belong to a single strongly connected dependency component.

The important methodological point is that profile control is layered on top of the existing formal machinery. The SAT encodings still certify solvability and binary cooperation. The profile analyzer uses those certified levels as input and acts as a classification and filtering layer for the generator.

4.1.4.6 Constrained Random Cooperative Generator

The constrained random cooperative generator combines the geometric filters of the constrained solvable generator with the binary cooperation test and optional profile filter of the random cooperative generator. In other words, it first avoids immediately degenerate geometries, then requires the surviving candidates to satisfy the same solver-based cooperation criterion.

This generator therefore targets the same formally certified output class as the random cooperative generator, but with a sampling distribution biased away from trivial failures. It is useful when the goal is not only to obtain cooperative levels, but to obtain them with fewer discarded samples.

4.1.4.7 Constructive Solvable Generator

The constructive solvable generator replaces blind sampling with a partial-by-construction layout. It reserves one disjoint horizontal or vertical lane per agent, places each start at one end of its lane and the corresponding exit at the other end, and only samples walls and lasers outside the reserved traversable lanes. Additional lasers are accepted only if their beam segments avoid the reserved cells. The solver still acts as the final verifier, but the sampling process is strongly biased toward jointly solvable instances.

4.1.4.8 Constructive Cooperative Generator

The constructive cooperative generator further specialises the constructive idea by planting a deliberate dependency pattern. One laser is placed so that a helper agent must truncate a beam of its own colour before another lane becomes traversable. The resulting candidate is then verified with the standard and strict SAT encodings. In the current implementation, this generator supports the `cooperative` and `asymmetric` profile filters. It is useful when one wants deliberately cooperative instances rather than merely sampling for cooperation and hoping to find it by rejection.

4.1.4.9 Summary

Generator	Construction Bias	Solvable	Cooperative
Random Solvable	Uniform random sampling	Yes (SAT check)	No
Constrained Random Solvable	Random + geometric rejection	Yes (SAT check)	No
Random Cooperative	Uniform random sampling	Yes (SAT check)	Yes (strict UNSAT)
Constrained Random Cooperative	Random + geometric rejection	Yes (SAT check)	Yes (strict UNSAT)
Constructive Solvable	Reserved agent lanes	Yes (SAT check)	No
Constructive Cooperative	Reserved lanes + planted dependency	Yes (SAT check)	Yes (strict UNSAT)

Table 1: Overview of the implemented generators and their guaranteed properties.

4.2 Part 2 — Results

The current experimental evaluation is intentionally focused. Its purpose is not yet to validate the full generator family, but to measure the effect of one modeling choice inside the SAT solver: the formulation of the agent-uniqueness constraint.

4.2.1 Experimental Question

The experiment asks whether the **global** and **local** movement formulations introduced in Section 4.1.1 differ materially in CNF size and runtime on levels of increasing complexity.

This is an important baseline question because the uniqueness constraint appears at every time step for every agent. A poor formulation therefore affects the full reduction, not only a negligible subpart of it. The protocol itself is defined in Section 4.1.2; the present chapter reports only the level set, the observed results, and their interpretation.

4.2.2 Benchmark Instances

Four levels of increasing size and complexity were used:

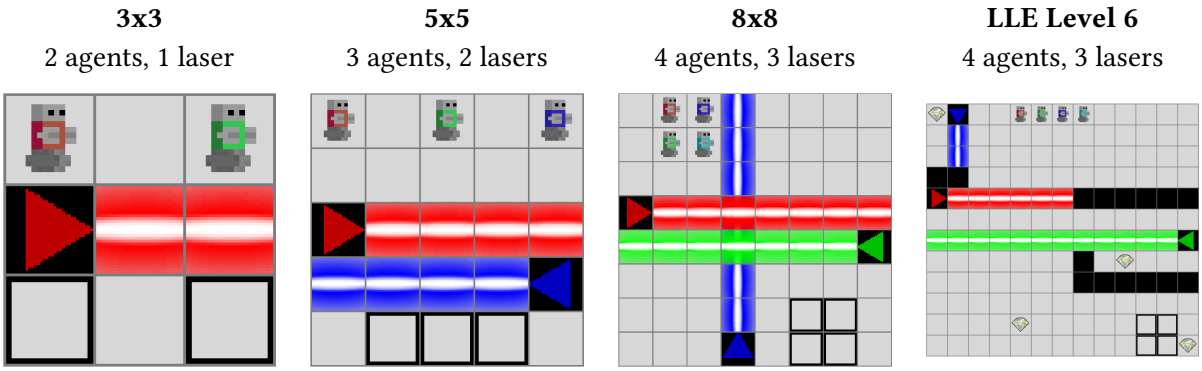

Figure 3: The four test levels used for benchmarking, ordered by increasing complexity.

The first three levels were constructed to expose scaling behaviour as grid size and agent count increase. The fourth is LLE Level 6, one of the hand-crafted benchmark levels distributed with LLE, included to assess solver behaviour on a realistic cooperative instance.

4.2.3 Results

4.2.3.1 Clause Count

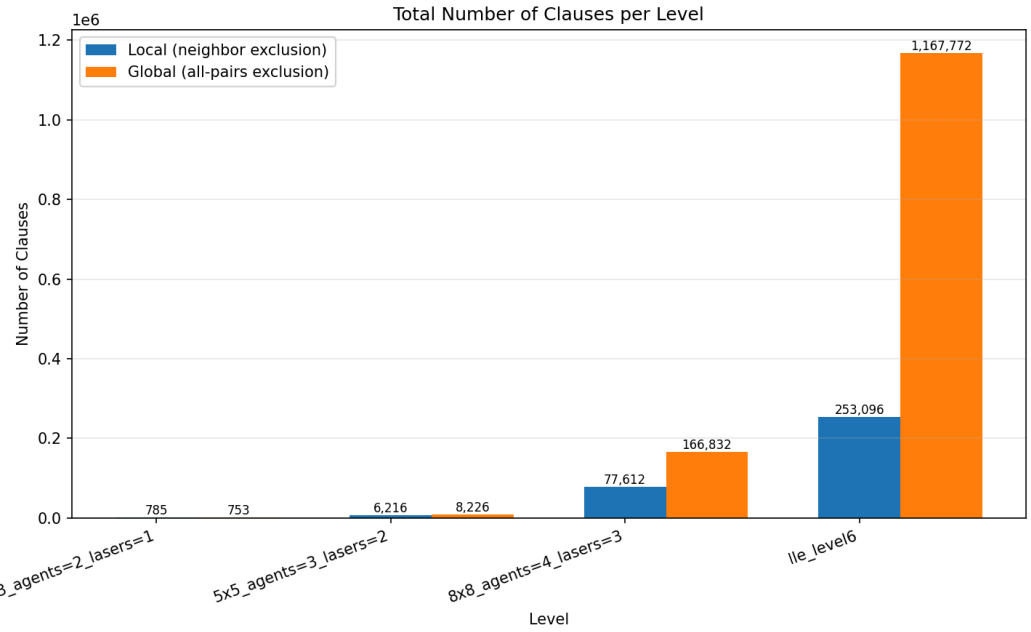

Figure 4: Number of clauses generated by the global and local uniqueness formulations across the four benchmark levels.

The clause-count results closely follow the theoretical expectation developed in Section 4.1.1. On the smallest 3×3 level, the global formulation is slightly more compact than the local one (753 clauses versus 785), which is consistent with the fact that the instance lies below the expected crossover regime. From the 5×5 level onward, the trend reverses: the local formulation generates 6,216

clauses versus 8,226 for the global one, then 77,612 versus 166,832 on the 8×8 level, and finally 253,096 versus 1,167,772 on LLE Level 6.

This widening gap is the main structural result of the experiment. The local formulation scales much better because its exclusivity clauses are tied to neighbourhood-sized successor sets, while the global formulation introduces pairwise exclusions across the full position set.

4.2.3.2 Solving Time

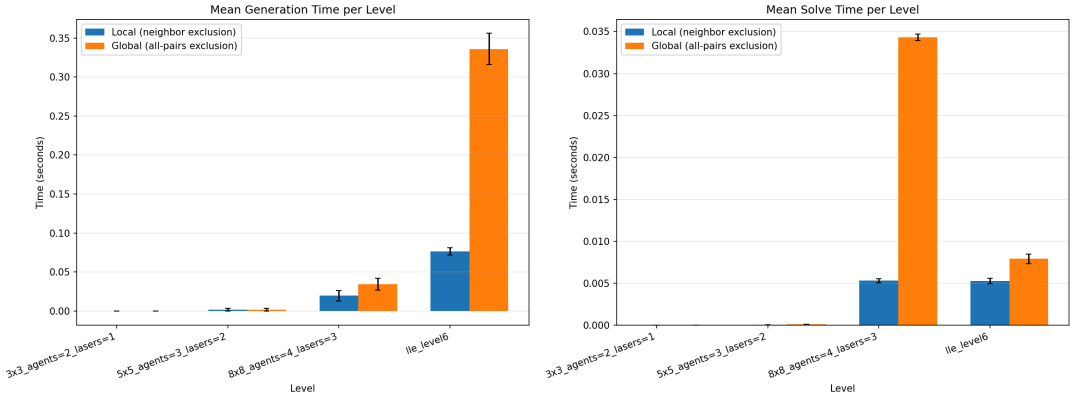

Figure 5: SAT solver time (seconds) for the global and local uniqueness formulations across the four benchmark levels.

The timing results show the same qualitative pattern. On the smallest synthetic instances, both methods solve essentially instantaneously and the difference is negligible. Once the levels become moderately large, however, the local formulation is consistently faster both to generate and to solve. The contrast is especially visible on the 8×8 level and on LLE Level 6, where the much larger global CNF induces a clear runtime penalty.

The practical conclusion is therefore the same as the analytical one: the local formulation is not merely a cleaner theoretical encoding, but the preferable default implementation choice for the solver developed in this thesis.

4.2.4 Interpretation and Scope

The current results support one claim strongly and several others only weakly.

What is established is that the internal SAT formulation matters. The choice between local and global uniqueness has a first-order effect on both CNF size and runtime, and that effect is already substantial on realistic LLE instances.

What is **not** yet established empirically is the broader quality of the generator framework. The present experiment does not measure generator acceptance rates, diversity of accepted levels, distribution of cooperation profiles, or any downstream training effect on MARL agents. The following sections address the first two of these open questions.

4.2.5 Generator Rejection Rates

4.2.5.1 Experimental Question

Acceptance rates determine the practical cost of the rejection-sampling strategy used by the generators. A generator with a 1 % acceptance rate requires approximately one hundred solver calls per usable level, which directly affects generation throughput. This experiment measures, for each generator and grid size, how many attempts are needed to find one accepted level and how much time those attempts consume.

4.2.5.2 Protocol

For each generator type and grid size combination, 50 trials are executed. Each trial consists of calling the generator with a budget of one attempt at a time until one level is accepted or a limit of 500 attempts is exhausted. Each single attempt samples a candidate layout, validates geometric constraints (where applicable), constructs an `lle.World` object, and runs the SAT solver. The attempt is accepted if the solver certifies the required property (solvability or cooperation), and rejected otherwise.

Recording the number of attempts per accepted level is the most direct measurement of the generator's rejection cost: the mean number of attempts is the reciprocal of the acceptance rate.

Four generators are evaluated: `constrained_random_solvable`, `constrained_random_cooperative`, `constructive_solvable`, and `constructive_cooperative`. The plain random generators are excluded as they serve primarily as structural baselines; the constrained and constructive variants are the generators of practical interest. Three grid sizes are used: 3×3 with 2 agents and 1 laser, 5×5 with 3 agents and 2 lasers, and 8×8 with 4 agents and 3 lasers.

4.2.5.3 Results

[Benchmark running — rejection_rate_by_generator]

Figure 6: Rejection rate (%) per generator and grid size, derived from the mean number of attempts per accepted level across 50 trials.

Results pending — run src/scripts/run_rejection_benchmark.py

Figure 7: Mean number of attempts needed to find one accepted level, per generator and grid size. Each attempt is one independent generation call; the mean is averaged over 50 successful trials.

[Benchmark running — time_per_accepted_level]

Figure 8: Mean wall-clock time (seconds) to obtain one accepted level, per generator and grid size. Time includes all rejected attempts that precede each accepted level.

4.2.5.4 Interpretation

The experiment is designed to quantify the overhead introduced by the SAT acceptance oracle. Solvable generators are expected to be rejected primarily because random layouts do not admit a valid joint trajectory within the requested horizon. Cooperative generators add the strict-semantics counterfactual check, which should increase rejection rates further: the set of layouts that are both solvable and cooperation-requiring is a strict subset of the solvable set.

The constrained generators are expected to improve acceptance rates by removing geometrically degenerate candidates before any SAT call is made. Their geometric pre-filter does not affect the formal guarantee: every accepted level still passes the same solver check. It only avoids spending solver time on candidates that fail for locally visible reasons.

The constructive generators are expected to show higher acceptance rates at small sizes because the lane-reservation strategy biases layouts toward solvable configurations by design. At larger sizes, the balance between reserved lanes and available free cells shifts, which may increase rejection rates for incidental reasons such as laser placement failures.

4.2.6 Cooperation Profile Distribution

4.2.6.1 Experimental Question

Binary cooperation detection certifies that every accepted cooperative level requires at least one same-colour beam-truncation step, but it does not distinguish which structural pattern of inter-agent dependency the level exhibits. This experiment measures the distribution of cooperation profiles among accepted levels, using the cooperation profile analyzer introduced in Section 4.1.3.

4.2.6.2 Protocol

For each of two cooperative generators (`constrained_random_cooperative` and `constructive_cooperative`) and two grid sizes (5×5 with 2 agents and 1 laser, 8×8 with 3 agents and 2 lasers), 100 cooperative levels are accepted and then classified by the cooperation profile analyzer. The classifier assigns each level to one of the following profile families: `asymmetric`, `mutual`, `chain`, `distributed`, or `fully_coupled`. The analyzer also returns `cooperative` as a fallback when cooperation is required but no specific dependency structure is detected.

4.2.6.3 Results

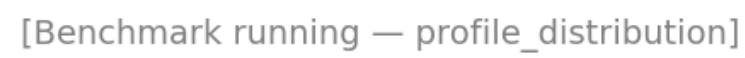

Figure 9: Distribution of cooperation profiles among accepted cooperative levels, grouped by generator and grid size. Each bar shows the fraction of accepted levels assigned to a given profile family.

4.2.6.4 Interpretation

The profile distribution is expected to reflect the structural biases of each generator. Random generators sample layouts without any structural preference for cooperation type, so their output distribution should be determined by how often different dependency patterns arise in the accepted subset of random layouts. Constructive generators, by contrast, plant a deliberate beam-blocking dependency in each candidate, which should bias the output toward simpler, structurally cleaner profiles.

This comparison is designed to serve two purposes. First, it tests whether the profile analyzer produces non-trivial and generator-dependent distributions — checking that the classifier does not return a constant answer. Second, it reveals which profiles are rare or absent under rejection sampling, motivating the profile-targeted generation mode in which the generator runs the cooperation profile check and accepts only levels that match a requested profile.

4.2.7 Discussion

The three experiments above characterise complementary aspects of the generation framework.

The rejection-rate experiment measures **efficiency**: how many solver calls are needed per accepted level, and which generator and grid-size combinations are most practical. This is relevant for large-scale generation tasks where throughput matters.

The profile-distribution experiment measures **output diversity**: whether the accepted levels cover different cooperation structures or concentrate on a narrow profile class. This is relevant for curriculum design, where a varied distribution of cooperation structures is preferable to a homogeneous one.

Together, the three experiments support the claim that the solver-in-the-loop architecture produces not only formally certified levels, but a characterisable and controllable distribution of certified levels.

What these experiments do not establish is whether certified levels are useful for training. The generator framework guarantees formal properties of accepted levels, but whether those properties translate into better or faster learning for MARL agents compared to uncertified baselines remains an open empirical question, addressed in Section 4.2.8.

4.2.8 Transfer to Human-Designed Levels

4.2.8.1 Experimental Question

The preceding experiments establish that the generator framework produces formally certified levels with controlled cooperation structure. They do not, however, address the downstream question of whether those levels are useful for training. This section addresses RQ4: can agents trained exclusively on procedurally generated levels transfer their behaviour to human-designed levels?

The test set used for evaluation is the set of hand-crafted levels distributed with the benchmark, which includes levels of varying complexity and cooperation structure. Transfer is measured by evaluating trained agents on those levels without any fine-tuning after training.

4.2.8.2 Protocol

Agents are trained using a cooperative MARL algorithm on a curriculum of levels produced by the generator framework. Training levels are sampled from the cooperative generators to ensure that every training instance requires the blocking-based cooperative interaction. After training, agents are evaluated on the benchmark's human-designed levels. Performance is reported as the fraction of episodes in which all agents reach their exit tiles within the time horizon.

4.2.8.3 Results

This section will be completed once the training experiments are run.

4.2.8.4 Interpretation

A positive transfer result would confirm that the formal cooperation guarantee translates into useful training signal: levels that are provably cooperative expose agents to the coordination structure they need to solve the benchmark levels. A negative result would indicate that the distribution mismatch between generated and human-designed levels is a limiting factor, motivating further work on domain-adaptive generation.

Conclusion

5.1 Summary

This thesis developed a SAT-based framework for reasoning about solvability and cooperation in a modeled subset of the Laser Learning Environment. We first formalised bounded-horizon solvability as a decision problem, then reduced it to satisfiability of a CNF formula. On top of that reduction, we introduced a strict beam-semantics counterfactual that turns the blocking-based cooperation mechanism of LLE into a second decision problem on the same level.

These decision procedures were then embedded inside a family of procedural generators. The resulting framework does not guarantee that generated levels are pedagogically optimal for MARL, but it does guarantee that accepted levels satisfy the formal properties checked by the solver. This is the central contribution of the thesis: procedural generation coupled to explicit certification rather than procedural generation guided only by heuristic plausibility.

The empirical evaluation covers several aspects of the framework. The SAT encoding comparison shows that the local uniqueness formulation is the preferable default: it yields substantially smaller CNF formulas and lower runtimes as instances grow. The rejection-rate and profile-distribution experiments characterise the practical cost of the acceptance oracle and the output diversity of the generator family. A transfer experiment is planned to evaluate whether agents trained on certified cooperative levels can generalise to human-designed benchmark levels.

5.2 Future Work

Several directions extend the present work naturally.

- **Parameter sensitivity and diversity.** The current acceptance-rate experiments use fixed agent and laser counts per grid size. A fuller characterisation would vary these parameters to locate the practical frontier at which rejection rates become prohibitive, and would measure diversity of accepted levels within a fixed parameter setting.
- **Richer cooperation metrics.** The cooperation profile taxonomy used here covers five dependency structures. Extensions could include synchronous cooperation (agents that must coordinate in the same timestep) and chain length as a difficulty axis. These richer targets are already defined in the cooperation profile notes; their generation and evaluation remain future work.
- **Model extension.** Incorporating gem tiles and void tiles into the formal model would expand the scope of the formal guarantees. Gems introduce an additional reachability condition; void tiles require a revised movement semantics. Both extensions preserve the SAT-based architecture.

- **Formal NP-hardness.** The present thesis shows that bounded-horizon LLE solvability is in NP and is reducible to SAT. Whether it is NP-hard — that is, whether there exists a polynomial-time reduction from a known NP-hard problem to LLE solvability — remains open and would be a worthwhile theoretical complement.

The main open question is therefore not whether solver-based certification is possible in LLE; the present thesis answers that positively. The open question is how far that certification framework can be extended — in terms of richer mechanics, richer cooperation structures, and downstream learning effects — before additional formal layers are required.

Appendix

A.3 Training Level Set

This appendix lists the levels used to train agents in the transfer experiment (Section 4.2.8). All levels were generated by the cooperative generator framework and certified by the SAT-based acceptance oracle. For each level, the grid size, agent count, laser count, and cooperation profile are reported.

To be completed: insert level images and parameter table here.

A.4 Benchmark Levels for SAT Encoding Comparison

The four levels used in the SAT encoding comparison (Section 4.2) are shown below with their exact parameters.

Level	Grid	Agents	Lasers	Horizon $T_{\max}$
Synthetic 3×3	3×3	2	1	8
Synthetic 5×5	5×5	3	2	12
Synthetic 8×8	8×8	4	3	32
Benchmark Level 6	varies	4	3	known

Table 2: Parameters of the four benchmark levels used in the SAT encoding comparison.

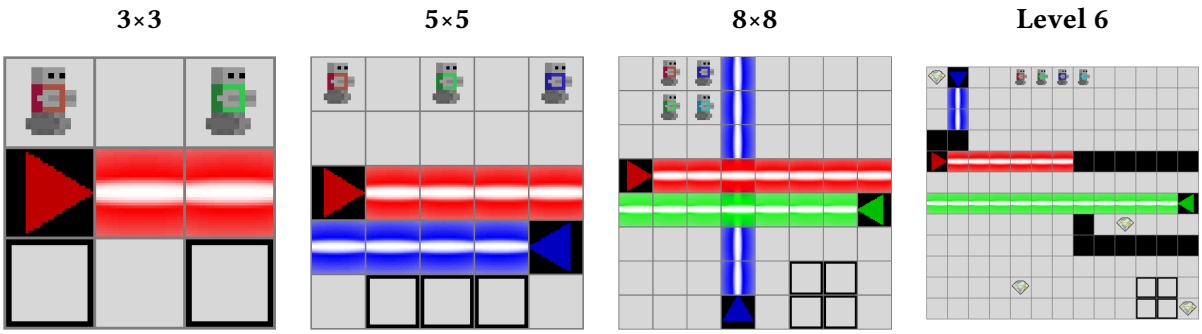

Figure 10: Visual representation of the four benchmark levels.

A.5 Generator Parameters

Parameter	3×3	5×5	8×8
Agents	2	3	4
Lasers	1	2	3
$T_{\max}$	8	12	20
Trials (rejection benchmark)	50	50	20
Max attempts per trial (rejection benchmark)	500	500	500
Levels generated (profile benchmark)	—	100	10

Table 3: Generator parameters used in the rejection rate and profile distribution benchmarks.

A.6 Cooperation Profile Examples

The cooperation profile analyzer classifies accepted cooperative levels into five structural families based on the inter-agent dependency graph. The definitions are given in Section 4.1.4. Representative visual examples of each profile type are provided below once the generator benchmarks are completed.

- **Asymmetric**: Agent A depends on Agent B, but not vice versa.
- **Mutual**: Agents A and B each depend on the other.
- **Chain**: Dependencies form a directed chain A $\longrightarrow$ B $\longrightarrow$ C.
- **Distributed**: One agent depends on multiple distinct helpers.
- **Fully coupled**: All agents belong to a single strongly connected dependency component.

To be completed: insert representative level images for each profile type.

Bibliography

[1] Y. Molinghen, R. Avalos, M. Van Achter, A. Nowé, and T. Lenaerts, "Laser Learning Environment: A New Environment for Coordination-Critical Multi-Agent Tasks," in *Artificial Intelligence and Machine Learning*, F. A. Oliehoek, M. Kok, and S. Verwer, Eds., Cham: Springer Nature Switzerland, 2025, pp. 135–154.

[2] N. Shaker, J. Togelius, and M. J. Nelson, *Procedural Content Generation in Games*. Springer, 2016. doi: 10.1007/978-3-319-42716-4.

[3] P. Surynek, "Problem Compilation for Multi-Agent Path Finding: A Survey," in *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence*, in IJCAI-22. 2022, pp. 5615–5620.

[4] M. Frommknecht and P. Surynek, "Solving Multi-Agent Pathfinding with Stochastic Local Search SAT Algorithms," in *Proceedings of the 21st International Conference on Informatics in Control, Automation and Robotics*, in ICINCO 2024, vol. 1. 2024, pp. 67–78. doi: 10.5220/0012944800003822.

[5] S. A. Cook, "The Complexity of Theorem-Proving Procedures," in *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, 1971, pp. 151–158.

[6] M. Davis, G. Logemann, and D. Loveland, "A Machine Program for Theorem-Proving," *Communications of the ACM*, vol. 5, no. 7, pp. 394–397, 1962, doi: 10.1145/368273.368557.

[7] A. Ignatiev, A. Morgado, and J. Marques-Silva, "PySAT: A Python Toolkit for Prototyping with SAT Oracles," in *Theory and Applications of Satisfiability Testing – SAT 2018*, in Lecture Notes in Computer Science, vol. 10929. Springer, 2018, pp. 428–437. doi: 10.1007/978-3-319-94144-8_26.

[8] M. L. Littman, "Markov Games as a Framework for Multi-Agent Reinforcement Learning," in *Machine Learning Proceedings 1994*, Morgan Kaufmann, 1994, pp. 157–163. doi: 10.1016/B978-1-55860-335-6.50027-1.

[9] L. S. Shapley, "Stochastic Games," *Proceedings of the National Academy of Sciences*, vol. 39, no. 10, pp. 1095–1100, 1953, doi: 10.1073/pnas.39.10.1095.

[10] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018. [Online]. Available: http://incompleteideas.net/book/the-book-2nd.html

[11] J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne, "Search-Based Procedural Content Generation: A Taxonomy and Survey," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 3, pp. 172–186, 2011, doi: 10.1109/TCIAIG.2011.2148116.

[12] P. Sunehag *et al.*, "Value-Decomposition Networks For Cooperative Multi-Agent Learning Based On Team Reward," in *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems*, in AAMAS 2018. International Foundation for Autonomous Agents, Multiagent Systems, 2018, pp. 2085–2087.

[13] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum Learning," in *Proceedings of the 26th Annual International Conference on Machine Learning*, in ICML 2009. ACM, 2009, pp. 41–48. doi: 10.1145/1553374.1553380.

[14] T. Rashid, M. Samvelyan, C. Schroeder de Witt, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning," in *Proceedings of the 35th International Conference on Machine Learning*, in ICML 2018, vol. 80. PMLR, 2018, pp. 4295–4304.